Universidad Mariano Gálvez de Guatemala · Desarrollo Web

PROYECTO FINAL · PRESENTACIÓN RESUMIDA

Sistema de Punto de Venta e Inventario

«Las Casas del Rincón Guatemalteco»

ESTUDIANTE
Andrés Ixcajoc López

CARNÉ
090-23-19115

CATEDRÁTICO
Jorge Mario Moya Marroquín

EL PROYECTO

Un POS web que controla todo el negocio

«Las Casas del Rincón Guatemalteco» vende productos de consumo diario (pastas, cárnicos, embutidos, lácteos, golosinas, aguas gaseosas y carbonatados) a los residentes de colonias. El negocio necesitaba **vender de forma ágil** y, al mismo tiempo, **saber qué mercadería entra, qué se vende y cuánto dinero ingresa o sale**.

La solución es una aplicación web que integra en un solo lugar la caja, el despacho con lector de código de barras, el cobro, la facturación con un código QR de promoción, las devoluciones, el inventario, los reportes y la conexión con el **Core de Empresas de Consumo Masivo**, el sistema que el negocio ya tenía.

Cajero
Abre y cierra la caja, despacha, cobra, factura y atiende devoluciones.

Administrador
Mantiene catálogos, compras, promociones, usuarios y consulta los reportes.

Cliente
Recibe su factura con QR y descubre su promoción desde el celular.

Core de Consumo Masivo
Sistema existente que provee productos, proveedores y precios.

REQUERIMIENTOS

Todo lo que pidió el enunciado, implementado

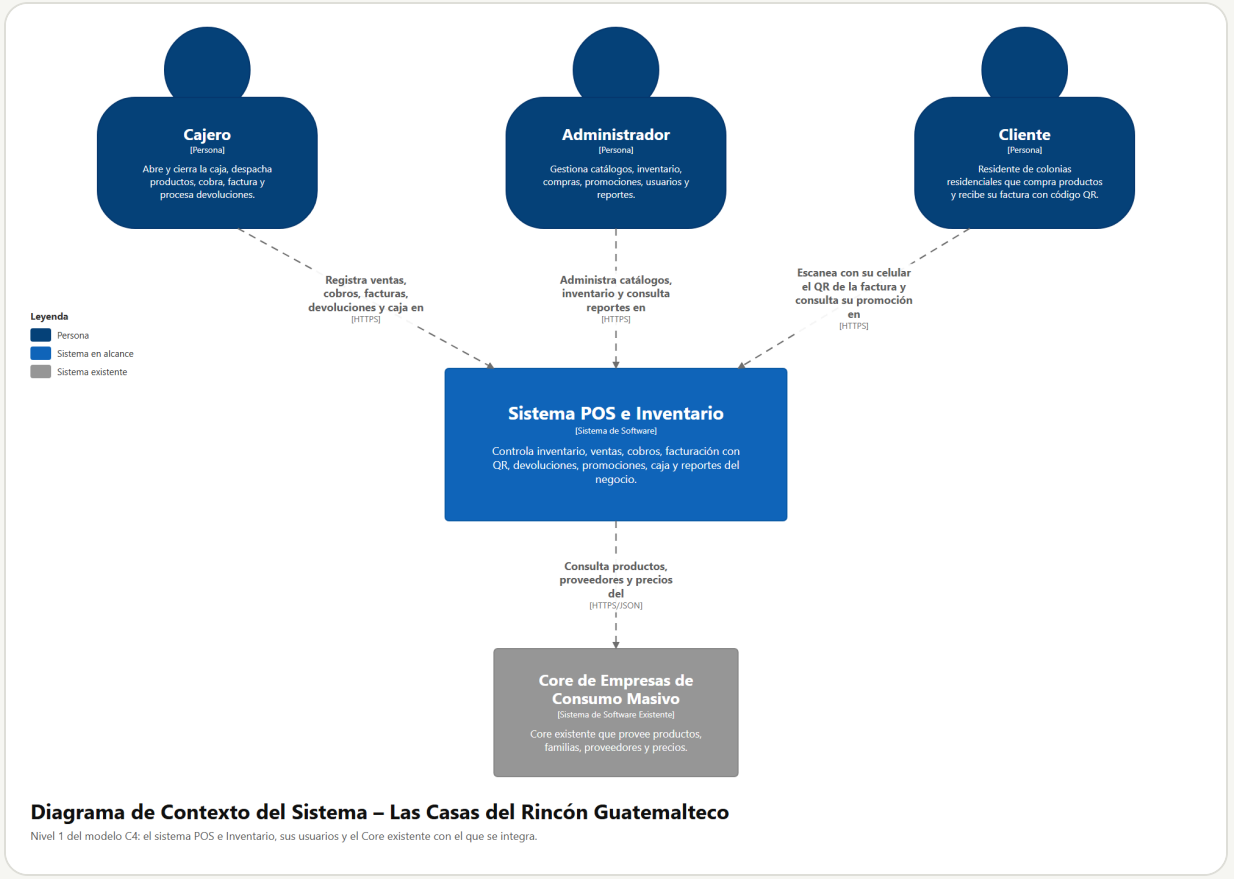
1	**Acceso**	Usuario y contraseña cifrada validados contra la base de datos, con roles y permisos.
2	**Catálogos**	Clientes, proveedores, productos por familia y precios normal y de mayoreo.
3	**Despacho**	Cada lectura del código suma a su línea; se muestran subtotal por línea y total.
4	**Devoluciones**	Sobre una factura, por daño o vencimiento; el producto sale como baja.
5	**Cobro**	Efectivo con cálculo de cambio, o tarjeta de crédito/débito con su voucher.
6	**Facturación**	Datos del cliente (NIT o CF), todos los productos, descuentos y total; también en PDF.
7	**Reportes**	Cierre del día (corte de caja), devoluciones, clientes y compras, en pantalla y PDF.
8	**Código QR**	Impreso en la factura; al leerlo con el celular muestra la promoción ganada.
9	**Menú interactivo**	Barra por módulos que muestra a cada rol solo lo que puede usar.
10	**Caja**	Apertura con monto inicial y corte con efectivo esperado, contado y diferencia.
10a	**Promociones**	Producto gratis, vale de regalo, 2.º a mitad de precio, mayoreo y puntos.
+	**Inventario y Core**	Compras, existencias y kardex; sincronización con el sistema existente.

MODELO 1 · C4

C4: de lo general a lo detallado

El modelo C4 describe el sistema como un mapa al que se le hace zoom. El **nivel 1 (contexto)** muestra el sistema completo, sus usuarios y el Core de Consumo Masivo. El **nivel 2 (contenedores)** muestra las piezas que se ejecutan: la interfaz web, la vista móvil del QR, la aplicación Laravel y la base de datos PostgreSQL.

El **nivel 3 (componentes)** abre la aplicación y muestra sus módulos (Caja, Despacho, Promociones...) y los adaptadores técnicos. El **nivel 4 (código)** llega a las clases del despacho. Así cualquiera entiende primero el panorama y luego el detalle, sin perderse.



Nivel 1 – Diagrama de contexto

MODELO 2 · CASOS DE USO

Casos de uso: qué hace cada actor

Un caso de uso es una acción completa que un actor realiza para lograr algo, como «Cobrar la venta» o «Leer el QR de la factura». Se elaboró un diagrama general y cinco por área: acceso y catálogos, punto de venta, devoluciones e inventario, promociones con QR y reportes.

Las relaciones aclaran el comportamiento: **«include»** marca un paso obligatorio (cobrar siempre registra el ingreso en caja), **«extend»** un paso opcional (canjear un cupón durante el despacho) y la **generalización** las variantes, como los tres métodos de pago o los cinco tipos de promoción.

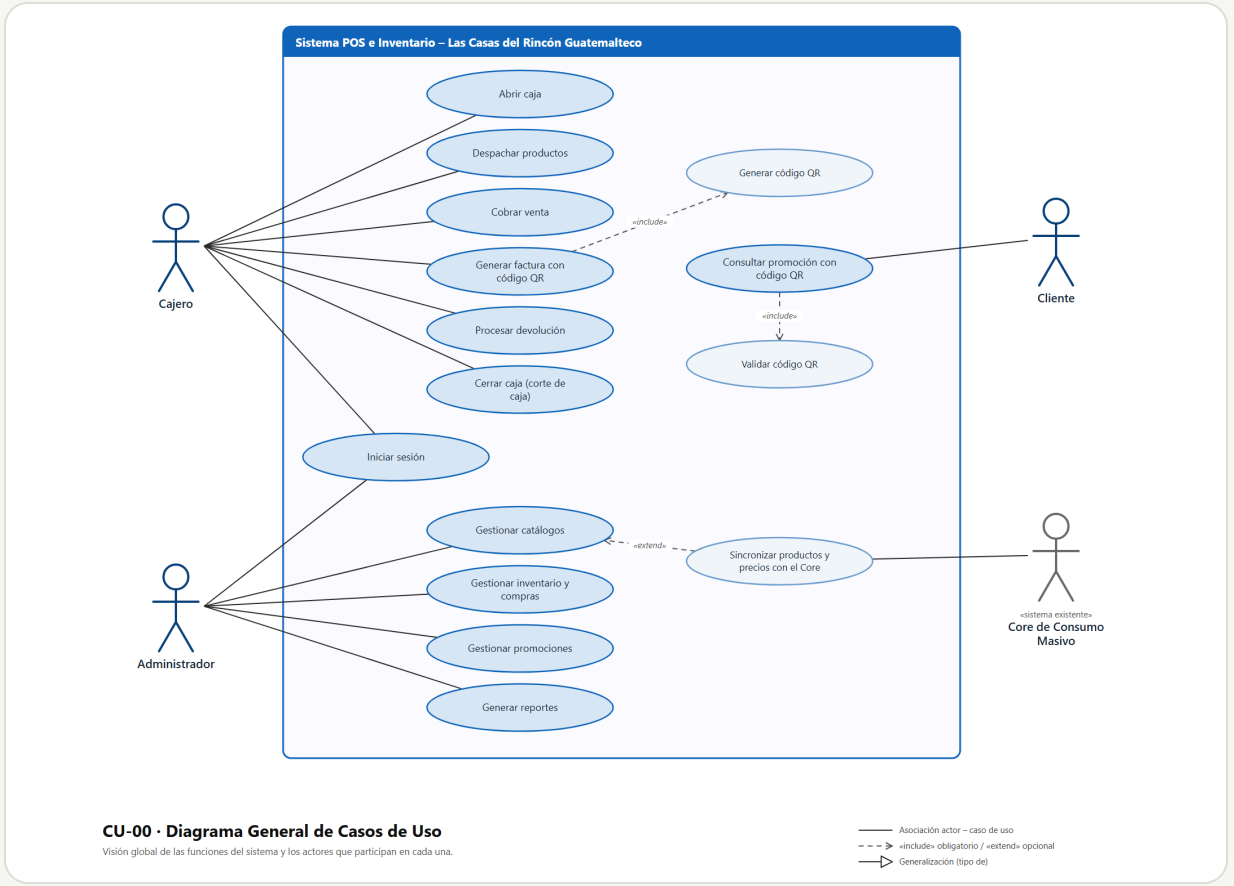


Diagrama general de casos de uso

MODELO 3 · DDD

DDD: el software organizado igual que el negocio

El diseño guiado por el dominio parte de una pregunta: *¿qué hace el negocio y cuáles son sus reglas?* El negocio se dividió en **10 contextos delimitados**, cada uno con su vocabulario y responsabilidades: Seguridad, Catálogos, Inventario, Caja, Despacho, Promociones, Cobros, Facturación, Devoluciones y Reportes.

Las reglas viven en el **dominio**, escrito en PHP puro y con las palabras del enunciado. Por ejemplo, la **Venta** (un agregado con sus líneas) impide vender más de la existencia, y el **MotorPromociones** aplica los descuentos. Así las reglas están en un solo lugar y no dependen de la base de datos ni del framework.

Principal
Despacho, Promociones, Cobros y Facturación: donde el negocio genera su valor.

Soporte
Catálogos, Inventario y Compras, Caja y Devoluciones: necesarios para operar.

Genérico
Seguridad y Accesos, y Reportes: comunes a cualquier sistema.

Lenguaje ubicuo
Clases como AbrirCajaUseCase, Venta::agregarProducto() o Caja::cerrar().

MODELO 4 · ARQUITECTURA HEXAGONAL

Hexagonal: el negocio separado de la tecnología

La arquitectura hexagonal funciona como un enchufe: el dominio define **puertos** (interfaces) con lo que ofrece y lo que necesita, y la infraestructura los conecta mediante **adaptadores**. El código se divide en tres capas: **Domain** (reglas), **Application** (casos de uso y puertos) e **Infrastructure** (web, PostgreSQL, QR, PDF y Core). Las dependencias siempre apuntan hacia el centro.

Cuando el cajero escanea un producto, el **DespachoController** llama al puerto **AgregarProductoUseCase**; la **Venta** valida la existencia, el motor aplica las promociones y el **repositorio Eloquent** guarda en PostgreSQL. Por eso el Core funciona por archivo o por API cambiando una sola línea del .env.



Vista general: dominio, puertos y adaptadores

MVC sobre Laravel y una base de datos relacional

El enunciado pide un modelo de desarrollo MVC, que Laravel aplica de forma natural y que convive con la arquitectura hexagonal. El **Modelo** son las reglas del dominio más los modelos Eloquent que leen y escriben las tablas; la **Vista** son las plantillas Blade de cada pantalla; y el **Controlador** recibe la petición del navegador y llama al caso de uso correspondiente.

La base de datos es **PostgreSQL 16**, creada con migraciones para que cualquier computadora tenga exactamente las mismas tablas. Las operaciones críticas usan **transacciones**: al cobrar se descuenta el inventario, se registra el pago y el ingreso en caja al mismo tiempo, o no se guarda nada.

Seguridad
usuarios, roles, permisos

Catálogos
clientes, proveedores, familias, productos

Inventario
compras, movimientos_inventario (kardex)

Ventas y caja
cajas, ventas, detalle_ventas, pagos, facturas

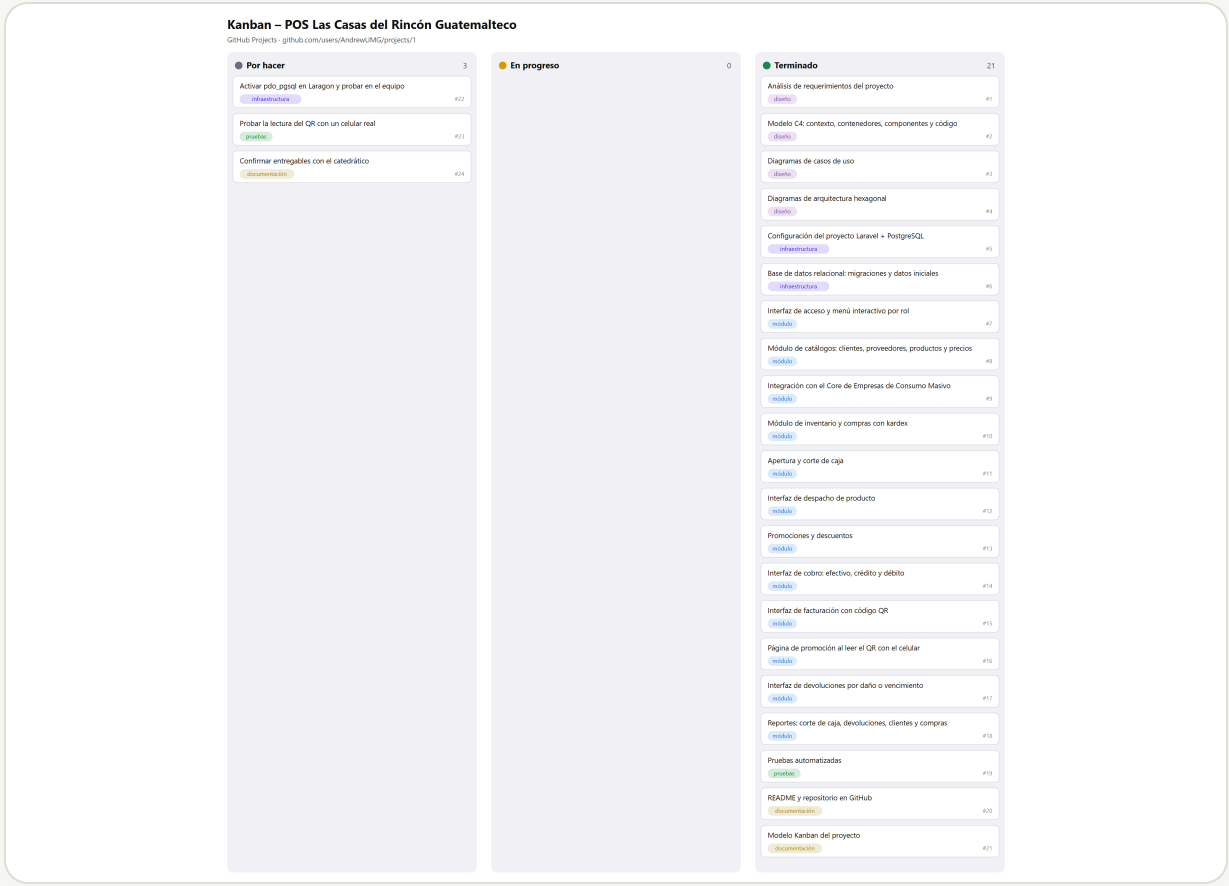
Promociones y devoluciones
promociones, cupones, devoluciones

MODELO 6 · KANBAN Y CALIDAD

Kanban y pruebas automáticas

El proyecto se organizó con un tablero **Kanban en GitHub Projects**. Cada tarea —diseño, módulos, infraestructura, pruebas y documentación— fue una tarjeta enlazada a un issue del repositorio, que avanzó de **Por hacer** a **En progreso** y a **Terminado**. Así siempre se supo qué faltaba y qué estaba listo.

Para asegurar que todo funciona se escribieron **17 pruebas automáticas con 147 verificaciones**: reglas del dominio, el flujo completo de una venta y todas las pantallas. Estas pruebas permitieron rediseñar la interfaz sin romper ninguna funcionalidad.



Tablero Kanban del proyecto

LA PLATAFORMA

Cómo funciona una venta de principio a fin

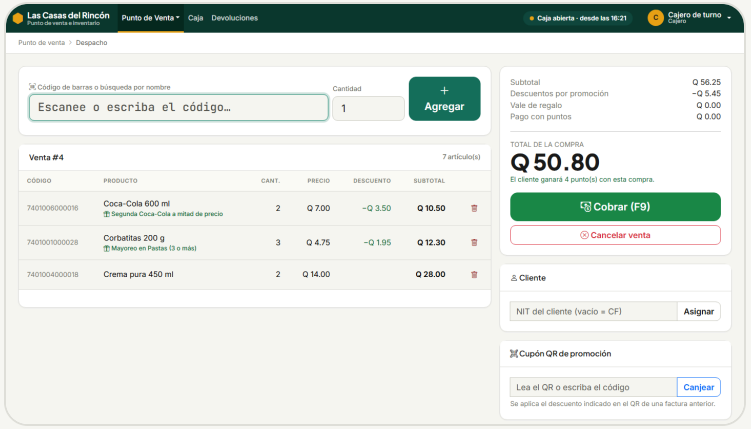
1 · Caja
El cajero inicia sesión y abre su caja con el monto inicial.

2 · Despacho
Escanea los productos; se aplican las promociones y se ve el total.

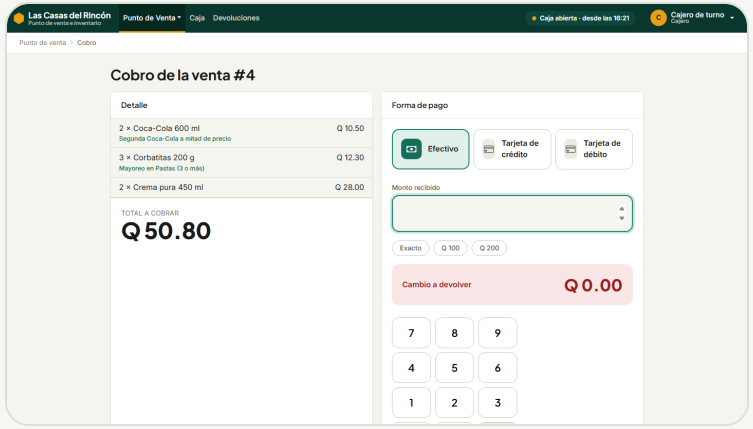
3 · Cobro
Efectivo con cambio, o tarjeta con su número de voucher.

4 · Factura
NIT o CF; la factura incluye un QR con una promoción.

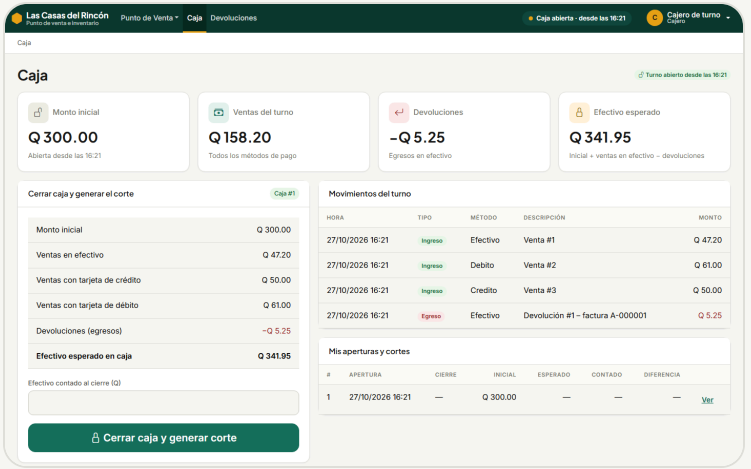
5 · Corte
Al final del turno compara el efectivo esperado con el contado.



Despacho por línea



Cobro con teclado y cambio



Caja y corte del turno

LA PLATAFORMA

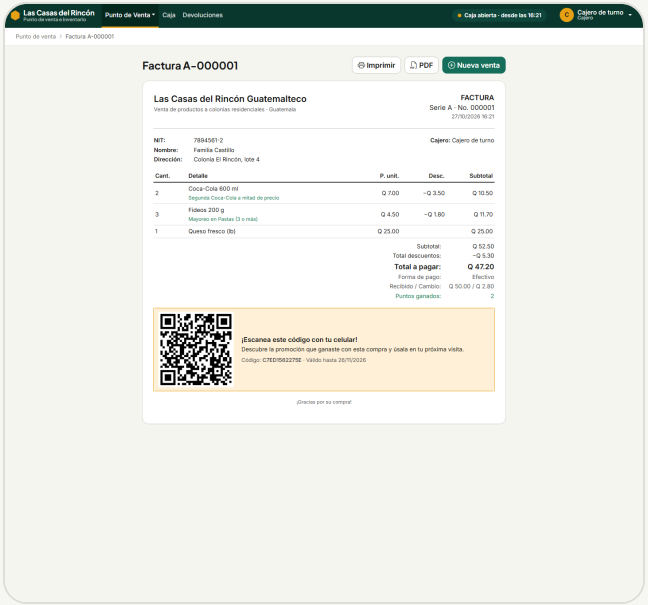
Promociones QR, inventario y Core

Al facturar, el sistema elige una promoción vigente y crea un **cupón único con vencimiento**. El QR de la factura abre una página pública donde el cliente ve su premio desde el celular, sin iniciar sesión. En su próxima compra el cajero canjea el código y el cupón queda usado. Las promociones automáticas se aplican solas.

El administrador registra las **compras** a proveedores, que aumentan las existencias, y consulta el **kardex**. Las **devoluciones** salen del inventario como baja y el reembolso sale de la caja. El catálogo se **sincroniza con el Core** y los **reportes** se obtienen en pantalla o PDF.



QR leído en el celular



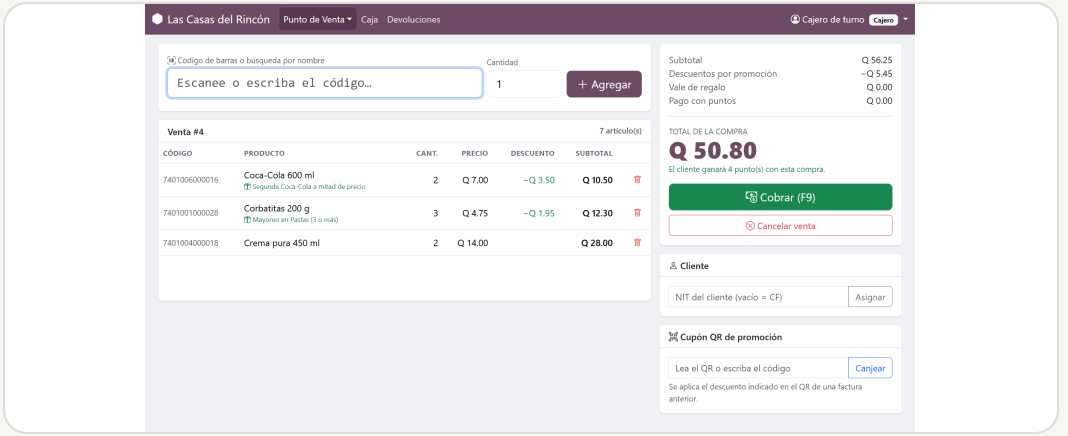
Factura con QR

DISEÑO DE INTERFAZ

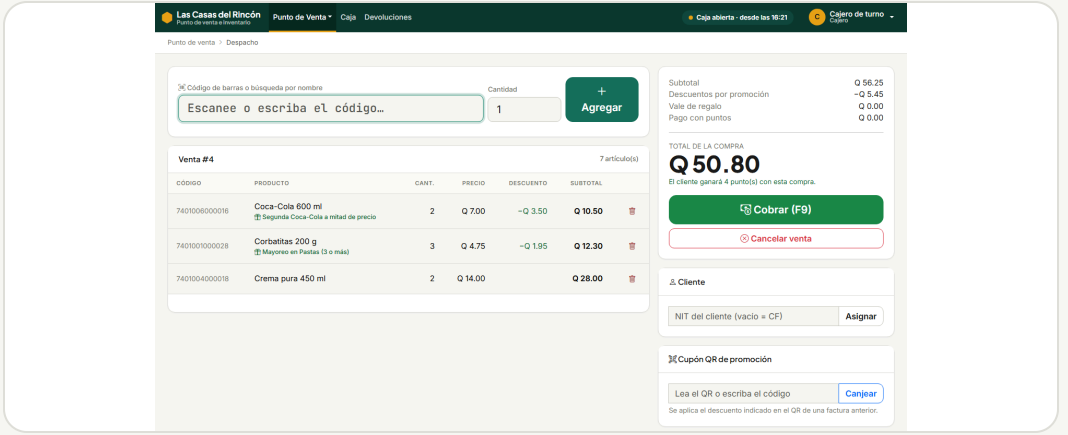
Una interfaz propia pensada para el cajero

Con el sistema funcionando se hizo una revisión de diseño: la primera versión usaba los colores de Odoo y una apariencia genérica. Se diseñó un **mockup completo en Figma** con más de 40 pantallas y un sistema de diseño propio, se aprobó y luego se aplicó al código.

Se conservó una estructura conocida (módulos arriba, migas, listas y fichas) con una identidad propia, «Jade & Maíz». Se agregaron un teclado numérico y montos rápidos para cobrar, el estado de la caja siempre visible, indicadores en el inicio y estados de color para el stock y las promociones.



Antes



Después · «Jade & Maíz»

CONCLUSIONES

- Se implementaron todos los requerimientos del enunciado, cada uno con su pantalla y verificado con pruebas automáticas.
- C4 y los casos de uso guiaron el diseño; DDD puso las reglas del negocio en un solo lugar y con su mismo lenguaje.
- La arquitectura hexagonal separó el negocio de la tecnología, y Kanban mantuvo el trabajo ordenado y visible.
- El sistema se instala en otra computadora con dos comandos y está documentado en el repositorio.

¡Gracias! ¿Preguntas?

https://github.com/AndrewUMG/pos-las-casas-del-rincon